

1.

```
1 import hashlib
2
3 file_name = '10k-most-common.txt'
4 target = "d54cc1fe76f5186380a0939d2fc1723c44e8a5f7"
5
6 num_to_char = {
7     'o': ['0', '0', 'o'],
8     '0': ['o', '0'],
9     'i': ['i', 'I', '1'],
10    '1': ['1', 'I', 'i'],
11    'I': ['1', 'i', 'I']
12 }
13
14 def recursive(word, start, stop):
15     if start == stop:
16         result_str = "".join(word)
17         hash_val = hashlib.sha1(result_str.encode('utf-8')).hexdigest()
18         if hash_val == target:
19             return result_str
20         else:
21             # print(f"{result_str}: {hash_val}")
22             return ""
23     org = word[start]
24     low = org.lower()
25     l = []
26     if low in num_to_char:
27         l = num_to_char[low]
28     elif low.isalpha():
29         l = [org.lower(), org.upper()]
30     else:
31         l = [org.lower()]
32     for c in l:
33         word[start] = c
34         result = recursive(word, start+1, stop)
35         if result != "":
36             return result
37     word[start] = org
38     return ""
39
40
41 found = False
42 with open(file_name, 'r') as f:
43     lines = f.readlines()
44     for w in lines:
45         word = w.strip()
46         end = len(word)
47         word = list(word)
48         result = recursive(word, 0, end)
49         if result != "":
50             print(f"found: {result}")
51             found = True
52             break
53 if not found:
54     print("Not found")
55
```

```
(comp_sec_ass1) (base) PS C:\Users\USER\Desktop\Comp_sec\ass1> python .\dict_attack.py
found: ThaiLand
(comp_sec_ass1) (base) PS C:\Users\USER\Desktop\Comp_sec\ass1> |
```

2.

Code:

```
1 import time
2 import random
3 import statistics
4 import hashlib
5 import bcrypt
6
7 file_name = '10k-most-common.txt'
8 with open(file_name, 'r') as f:
9     lines = f.readlines()
10 lis = []
11 for w in lines:
12     word = w.strip()
13     lis.append(word)
14
15 sample_sha1 = []
16 sample_md5 = []
17 sample_bcrypt = []
18 sample_list = random.sample(lis, 100)
19 for word in sample_list:
20     start = time.perf_counter()
21     hash_val = hashlib.sha1(word.encode('utf-8')).hexdigest()
22     end = time.perf_counter()
23     sample_sha1.append(end-start)
24
25     start = time.perf_counter()
26     hash_val = hashlib.md5(word.encode('utf-8')).hexdigest()
27     end = time.perf_counter()
28     sample_md5.append(end-start)
29
30     start = time.perf_counter()
31     salt = bcrypt.gensalt()
32     hash_val = bcrypt.hashpw(word.encode('utf-8'), salt).hex()
33     end = time.perf_counter()
34     sample_bcrypt.append(end-start)
35
36 q1, q2, q3 = statistics.quantiles(sample_sha1, n=4)
37 iqr = q3-q1
38 filter_time = [t for t in sample_sha1 if t >= q1-1.5*iqr and t <= q3+1.5*iqr]
39 print(round(sum(filter_time) / len(filter_time), 6))
40
41 q1, q2, q3 = statistics.quantiles(sample_md5, n=4)
42 iqr = q3-q1
43 filter_time = [t for t in sample_md5 if t >= q1-1.5*iqr and t <= q3+1.5*iqr]
44 print(round(sum(filter_time) / len(filter_time), 6))
45
46 q1, q2, q3 = statistics.quantiles(sample_bcrypt, n=4)
47 iqr = q3-q1
48 filter_time = [t for t in sample_bcrypt if t >= q1-1.5*iqr and t <= q3 + 1.5*iqr]
49 print(round(sum(filter_time) / len(filter_time), 6))
50
51
52
53
54
```

Answer:

```
(comp_sec_ass1) (base) PS C:\Users\USER\Desktop\Comp_sec\ass1> python .\measurement.py
1.6e-05
6e-06
0.382539
```

จะเห็นว่า bcrypt ใช้เวลานานกว่า sha1 และ md5 เป็น 23908 เท่าและ 63755 เท่าซึ่งใช้การหลักการทาง data sci โดยตัด outlier และ คัด trimmed mean ออกมาเป็นเวลาเฉลี่ย

3. วิธีทำ

โดยทำการหาเวลาใน 1 ปีเป็นหน่วย วินาทีก่อนจะได้ $365 * 24 * 60 * 60 = 31536000$ วินาที

และเนื่องจาก character ที่เป็นไปได้ จาก upper char, lower char, number, special char เป็น 97

ตัวอักษรอ้างอิงข้อมูลในห้องเรียนที่อาจารย์พูด

และ Assume ว่า การ brute force จะเจอ password ที่กึ่งกลางของ combination ทั้งหมดที่เป็นไปได้

จะได้ว่า $97^n * C / 2 \geq 31536000$

โดย n คือความยาวของ password และ C คือ ค่าเฉลี่ยของเวลาในการ hash password 1 ครั้ง

โดย C ขอ Assume ว่าเราไม่รู้ค่า hash password ด้วย algorithm ใดดังนั้นจะคิดแยกในแต่ละ algorithm

สำหรับ Sha1: $97^n * 0.000016 / 2 \geq 31536000$

จะได้ $n \sim 6.33979 \rightarrow n \geq 7$ ตัวอักษร

สำหรับ Md5: $97^n * 0.000006 / 2 \geq 31536000$

จะได้ $n \sim 6.55419 \rightarrow n \geq 7$ ตัวอักษร

สำหรับ bcrypt: $97^n * 0.382539 / 2 \geq 31536000$

จะได้ $n \sim 4.13594 \rightarrow n \geq 5$ ตัวอักษร

4. ตอบ เป็นไปได้แต่ไม่ practical เพราะ bcrypt algorithm มีการกำหนดคุณลักษณะของ password ไว้

ชัดเจนทำให้เราสามารถรู้ salt ที่เขาเติมได้จาก hash ที่ได้มาจากนั้นพอรู้ hash ก็ทำเพียงแค่ brute force ในส่วนของ password plain text จริงๆแล้วเอา salt ที่ได้จาก hash database มาเติมเพื่อทำ brute force เช็คต่อไป แต่ถึงกระนั้น bcrypt ก็ยังมีการหน่วงเวลาด้วย cost factor ทำให้การ hash password จะใช้เวลามากกว่า md5 หรือ sha1 มหาศาลทำให้ไม่มีประสิทธิภาพ

5. ตอบ ไม่ได้และไม่ practical เพราะ bcrypt ถูก design ให้เติม salt ที่มี characteristic เฉพาะทำให้การ precompute password อื่นๆมาก่อนไม่สามารถรู้ salt ได้ทำให้แม้จะ hash ได้ค่าออกมาก็จะไม่ตรงเพราะ salt ที่เติมไม่ตรงกัน ทำให้เป็นไปไม่ได้โดยแม้แต่การจะสร้างทุกความเป็นไปได้ของการ generate password พร้อมกับการ generate salt แล้วเก็บลง table ยิ่งเป็นไปไม่ได้และไม่มีประสิทธิภาพมหาศาล

6. ตอบ ใช้หลักการของ $3A + 2I$ ได้มัยโดยที่ผมจะให้ตัว โดยนำ Authentication มาระบุว่า user ที่ส่ง request access ข้อมูลมานั้นเป็นใครผ่าน credential ที่เขามี จากนั้น Authority ว่า User คนนั้นสามารถทำอะไรได้บ้าง เช่นอ่านข้อมูลของตัวเองได้อย่างเดียว โดยที่ผมจะกำหนดว่าจะไม่มีการ return password ที่ hash กลับไปให้ผ่าน request นั้นเพื่อป้องกันการหลุดของ password จากนั้น accounting ทำการเก็บ log ว่ามี user คนนี้มาทำการขอ access ข้อมูลหรือมาทำอะไรกับฐานข้อมูลของระบบ โดยเก็บทั้งเวลา task ที่เขาทำ และผมจะใช้ integrity ในการออกแบบ database ไม่ให้ถูกเจาะผ่านทางอื่นที่ไม่ได้กำหนดไว้ให้ เช่นการ DDOS ใส่ service ที่ทำหน้าที่ authenticate request ไปยัง database เพื่อให้ต้องผ่านการ authenticate ปกติทุกครั้ง โดยการกำหนด rate limit และ ผมจะใช้ Input validation ด้วย โดยการทำการป้องกัน sql injection เพื่อป้องกันผู้ไม่ประสงค์ดีดึงข้อมูล user ผู้อื่นออกไปได้

โดย hash function ผมจะใช้ bcrypt ที่ gen salt ด้วยข้อมูล algorithm random เป็นต้น เพื่อป้องกัน rainbow attack และ schema ของ database ก็ต้องกำหนดให้ password มีความยาวของ plaintext เกินกว่า 15 ตัวอักษรขึ้นไปเพื่อป้องกัน brute force อีกทั้งต้องมีการกำหนดให้ password มี lower char, upper char, number, special char ด้วยเพื่อป้องกัน dict attack โดยในมุมของ cost มองในแง่ของ performance โดยที่ถ้าเรากำหนดให้ การ process password ด้วยระบบดังกล่าวใช้เวลาไม่เกิน 500ms โดยการปรับ cost factor ซึ่ง user แทบจะไม่เห็นความแตกต่าง ด้วยเวลานี้มันสามารถที่จะทำให้ hacker ใช้เวลามากขึ้นในการนำ hash ที่เขามีมายิง request check ซึ่งใช้เวลามากเมื่อเทียบกับ possibility ของ combination password ที่เป็นไปได้ทำให้ cost ในแง่ของ performance ค่อนข้าง